

Refactor of Clustering Code in GEMini Plugin

Dean Reiter

DarkLight Software Meeting

September 14, 2026

1. Need for improved clustering method
2. Ideas for new clustering methods
3. Refactoring GEMini

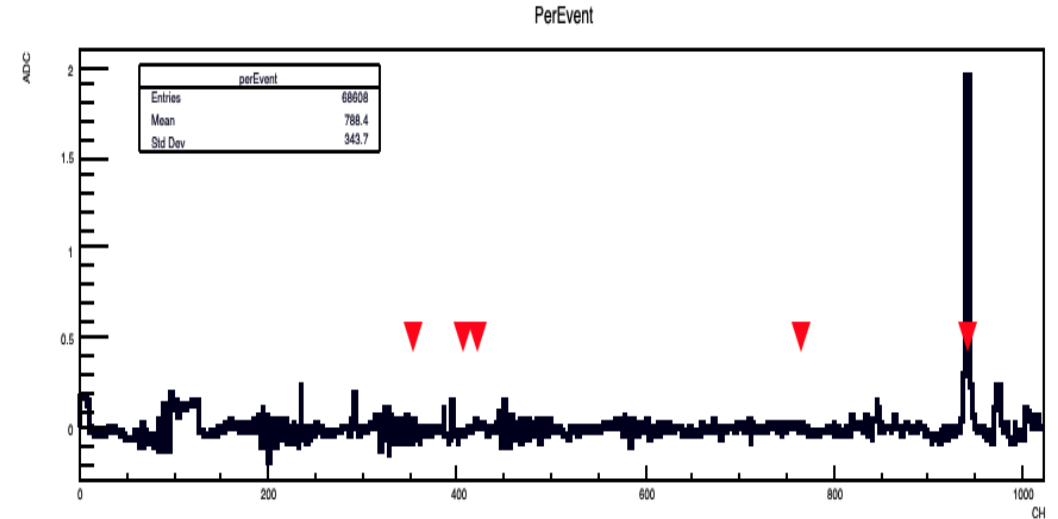
- Assumes cluster ADC signal is gaussian function of channel
 - Not optimal for signals wider or smaller than pre-defined width
 - Not optimal for detecting angled tracks
- Does not look at time-dependence of signals
- Small errors in implementation
 - e.g. signal convolution
- Incomplete
 - Cluster width, size, charge, time not well defined

Old Clustering Method – Example

3

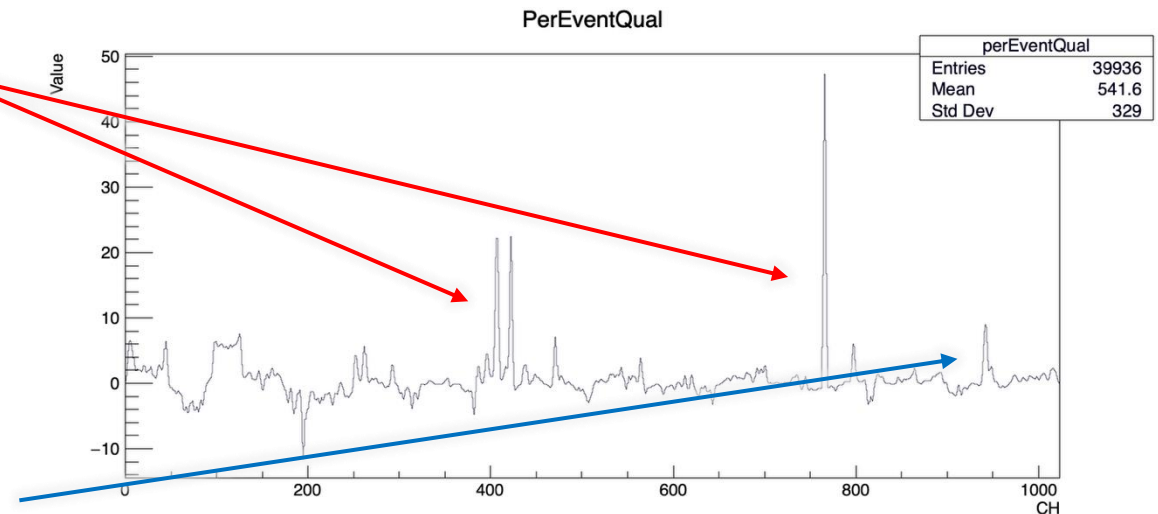
- If window of ADC values has same shape as gaussian template, “gaussian quality” → infinity
- Thresholds could be lowered

Time-averaged ADC values



Blows up noise which resembles
pre-defined gaussian template

“Gaussian quality”



Run 4010 (Elastic)
Event 12, Left Bottom X

Diminishes cluster signal

- We can gauge accuracy of cluster reconstruction with *Matching Efficiency*
 - i.e. what fraction of clusters can be spatially matched to trigger hits during track reconstruction
 - Or conversely, what fraction of trigger hits which can be matched to tracks
 - Spatial match = track candidate intersects trigger hit (within hit box)
 - Track candidate = pair of 2D clusters in bottom GEM and top GEM
- Upper limit on matching efficiency from detector acceptance, detector efficiency, etc.

Track Reconstruction Spatial Matching Efficiency Gaussian Quality Clustering Method Run 4005 (100k Events)		
	Left (Elastic)	Right (Moller)
X-Axis Clusters to In-Time Trigger Hits	1.7%	5.9%
Y-Axis Clusters to In-Time Trigger Hits	3.1%	8.9%
All 1D Clusters to In-Time Trigger Hits	1.7%	6.0%
Track Candidates to In-Time Trigger Hits	15%	13%
In-time Trigger Hits to Track Candidates	1.7%	5.9%

A few ideas...

1. Simple peak finder

- Find peaks by analyzing spatial derivatives of channel values

2. Time-domain first approach

- First identify channels with expected time-domain signal then find spatial groupings of those channels

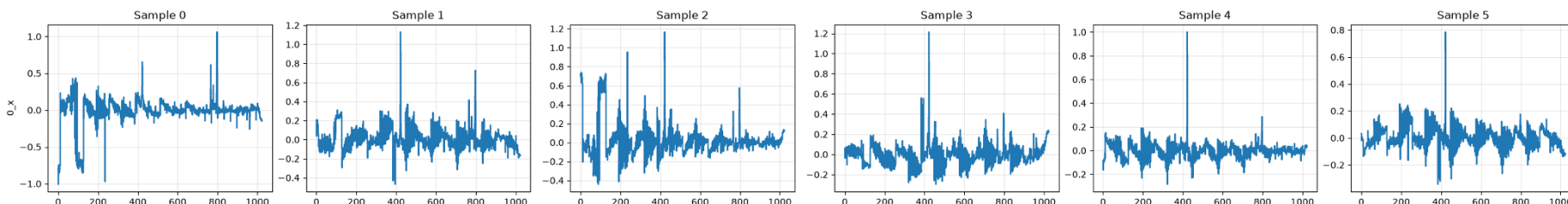
3. Matched filter

- Compute cross-correlation (like a convolution) of data with expected signal
- Amplifies occurrences of pre-defined template signal

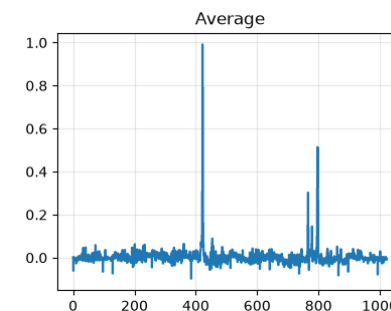
4. Continuous wavelet transform

- Transform data into σ -space, e.g. wavelet width space
- In practice, this is like computing many matched filters across spectrum of templates

1. Average channel values over time



1



2. Smooth data to eliminate noise fluctuations

- In this example, gaussian kernel smoothing

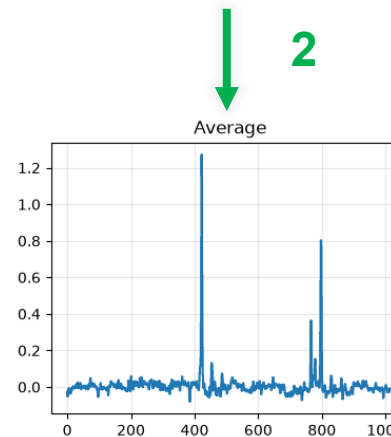
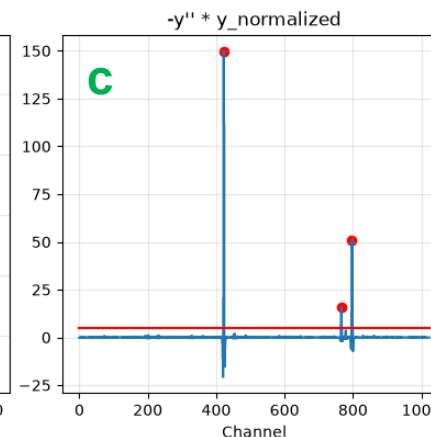
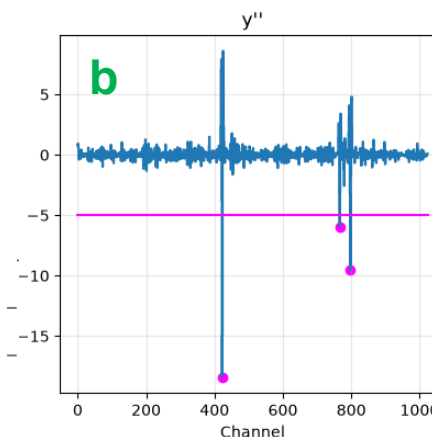
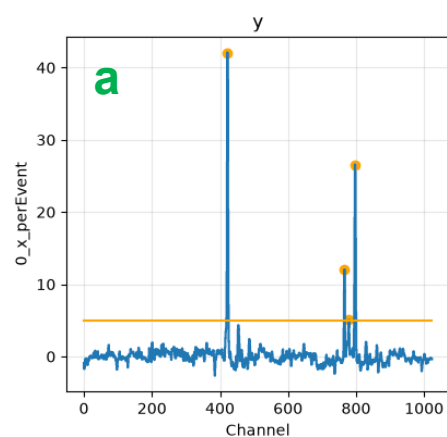
3. Find clusters

a. Maxima of y

b. Minima of $\frac{d^2y}{dx^2}$

c. Maxima of $-\frac{d^2y}{dx^2} y$

(5 std above mean threshold)



3

1. GEMapv corrects ADC value noise (common-mode, pedestal)
2. GEMapv condenses ADC values by taking average over time samples
3. GEMaxis stores condensed ADC values into large array
4. GEMaxis::process() finds clusters and stores them in gemhit structures
5. GEMplane saves cluster information as GEMhit objects in output tree

1. User can choose clustering method in Gemini.ini
 - Does not require recompile

```
readmode="AHEAD"  
clusterfinder="DEFAULT"
```

2. Cluster finding and analysis separated into modular functions

```
find_clusters();  
process_clusters();
```

- GEMaxis::find_clusters() finds and stores positions of clusters

```
//cluster position info from cluster finder, used to further analyze cluster properties  
struct cluster{  
    double loc, loc_err; //precise position of cluster peak [channel index] and uncertainty thereof  
    int start, end; //start (inclusive) and end (exclusive) positions of cluster [channel index]  
};
```

- GEMaxis::process_clusters() analyzes cluster properties and stores info in gemhit structures
 - Further separated into functions like get_amp1() and get_charge()
3. GEMaxis stores ADC values from all time samples – condensing ADC values is an optional step in cluster finder

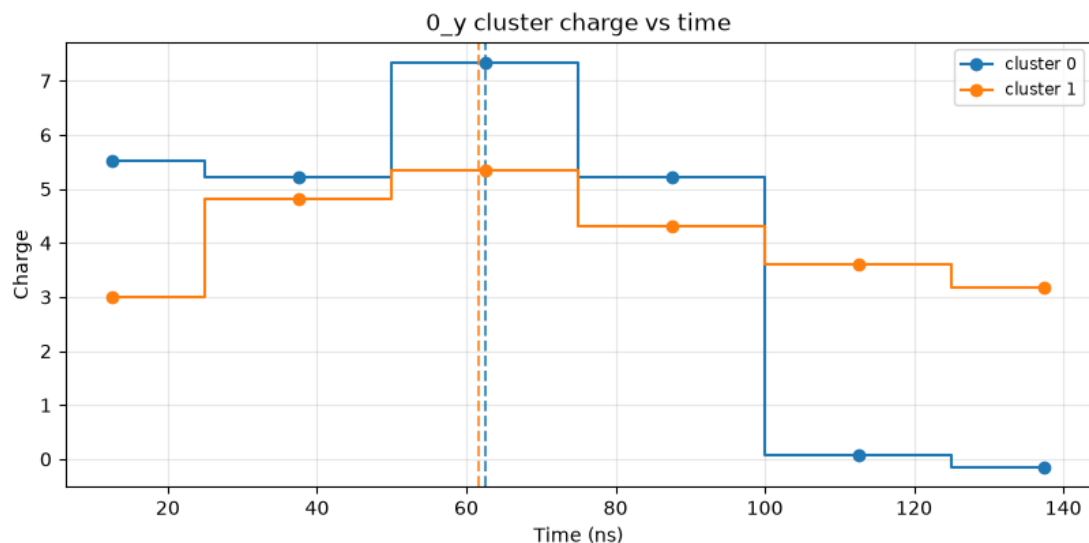
4. User can set cluster finder parameter values in command line
 - Not yet implemented
 - Allow for tuning of thresholds, cutoffs, etc. in cluster finding

```
cooker GEMini.ini input.root output.root -c GEMini:SetParams:5,0.98,6,0.98
```

5. Add more inputs for track reconstruction to output tree

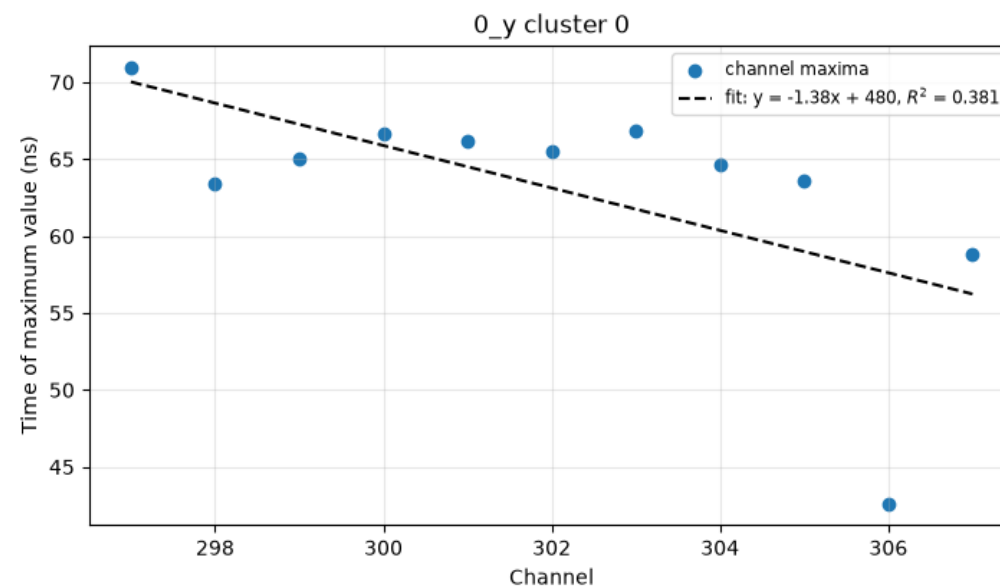
a) Cluster timing information

- “Hit time”
- Cluster size/charge profile over time
- All cluster ADC values over time



b) Track angle information

- Can do time-projection analysis of cluster and estimate track angle



Supposedly, $|\text{slope}| = (0.4\text{mm}) \cdot \frac{\cot \theta}{v_d} \sim 6.9$

- Draft pull request: <https://github.com/DarkLight-EXP/darklight-cooker/pull/237>
- Settings such as clustering method string and parameter values can be initially set by .ini file and overwritten by command line function call
- Example of how to pass string argument to cooker in command line:
`-c muonDecay_out:setOutputFile: '"sim_validation_decay.txt"'`

Backup

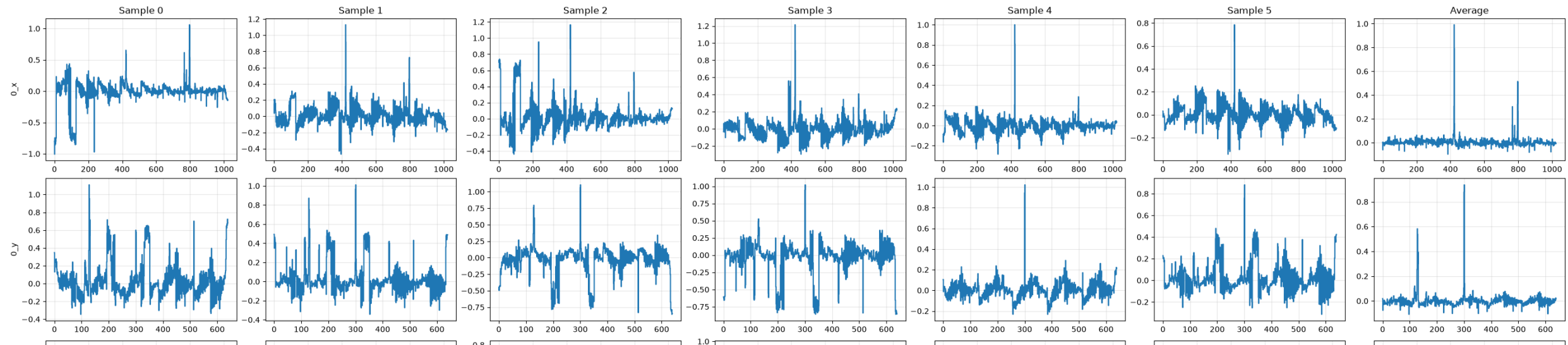
Tests with Run 4010, Event 5 (elastic)

- Time-sample averaging (samples 3-5 versus all samples)
- Channel smoothing (Savitzky-Golay, moving average, triangular and Gaussian kernels)
- Second derivative threshold peak detection
- Gaussian derivative matched filter
- “Gaussian quality”

Preferred cluster identification algorithm (based on initial tests)

1. Average over all time samples
2. Channel smoothing with 5-channel Gaussian kernel
3. Peak detection in $-y''$ multiplied by y (averaged, smoothed, and zeroed)
 - Threshold of 5 standard deviations above noise (all values $< 99^{\text{th}}$ percentile)

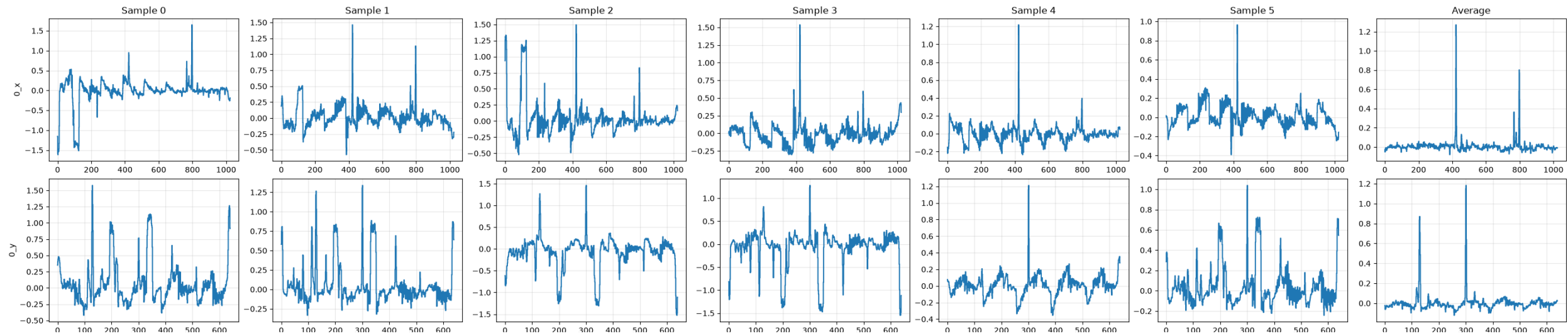
Left bottom GEM
Run 4010, Event 5

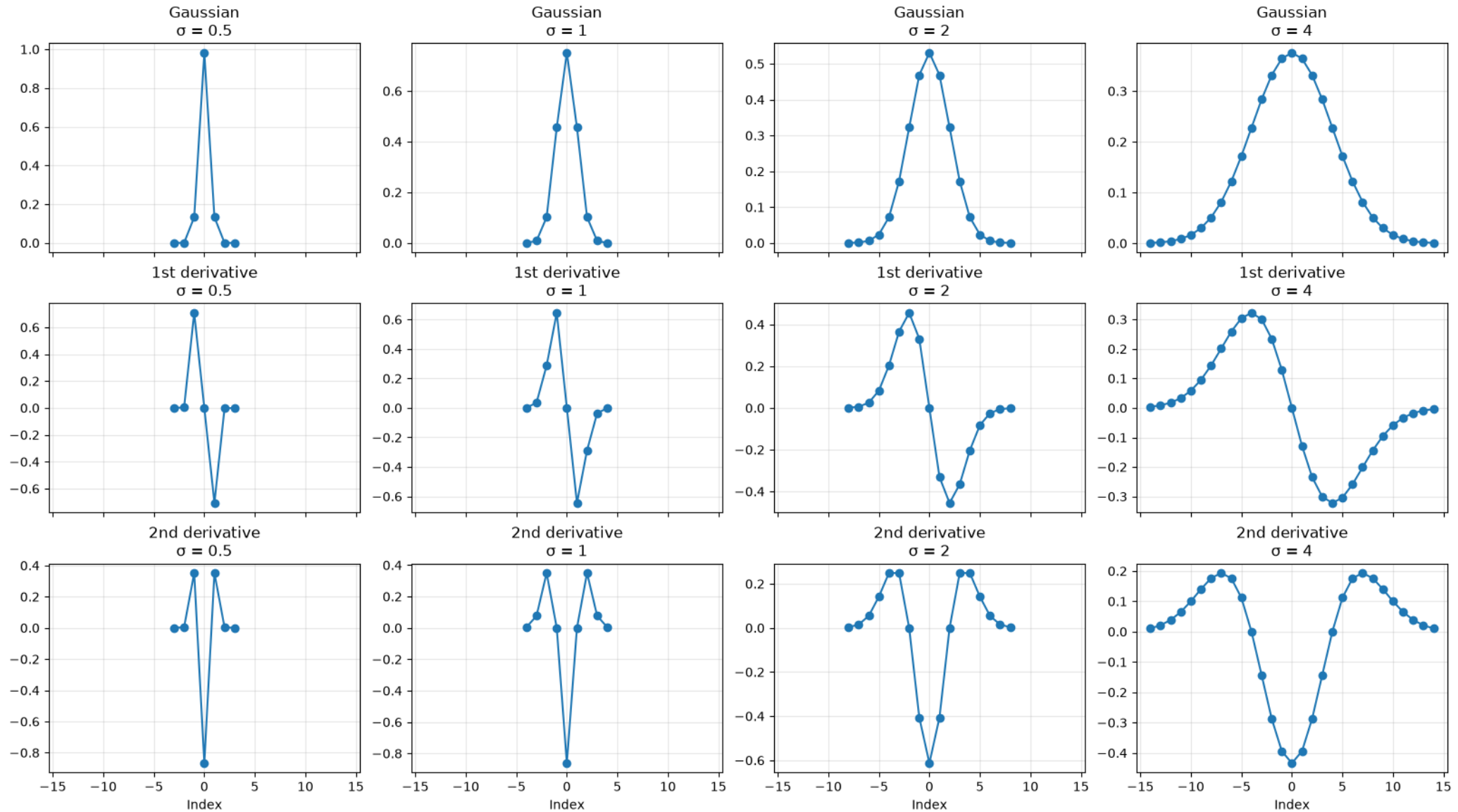


After applying smoothing

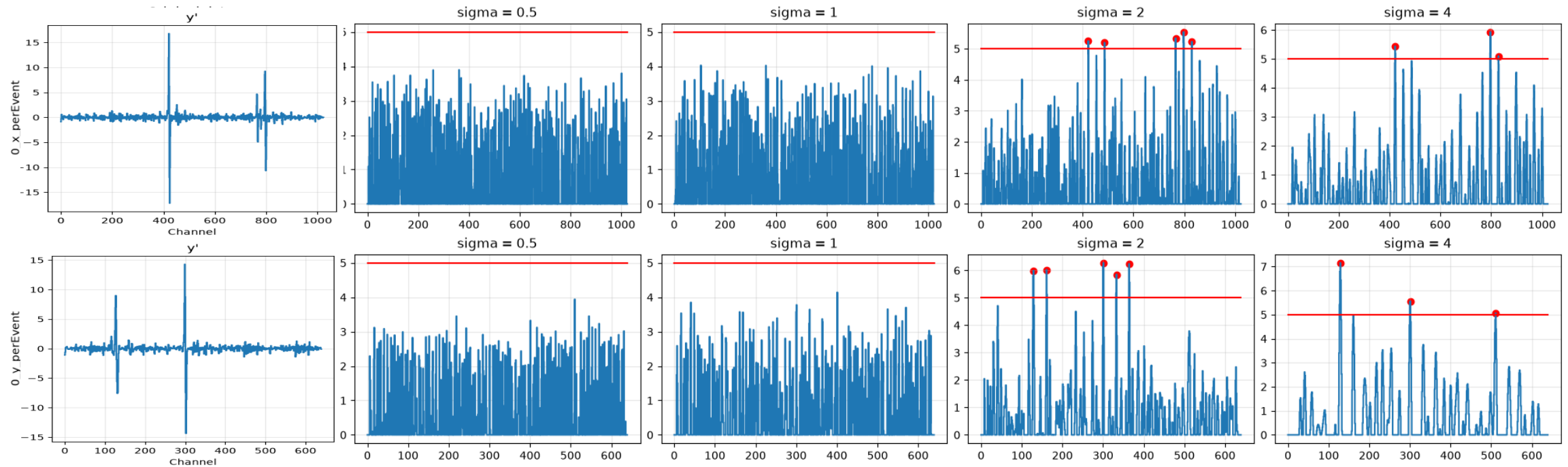
15

Smoothed with Gaussian kernel (sigma=1 channel, 9 channels wide)





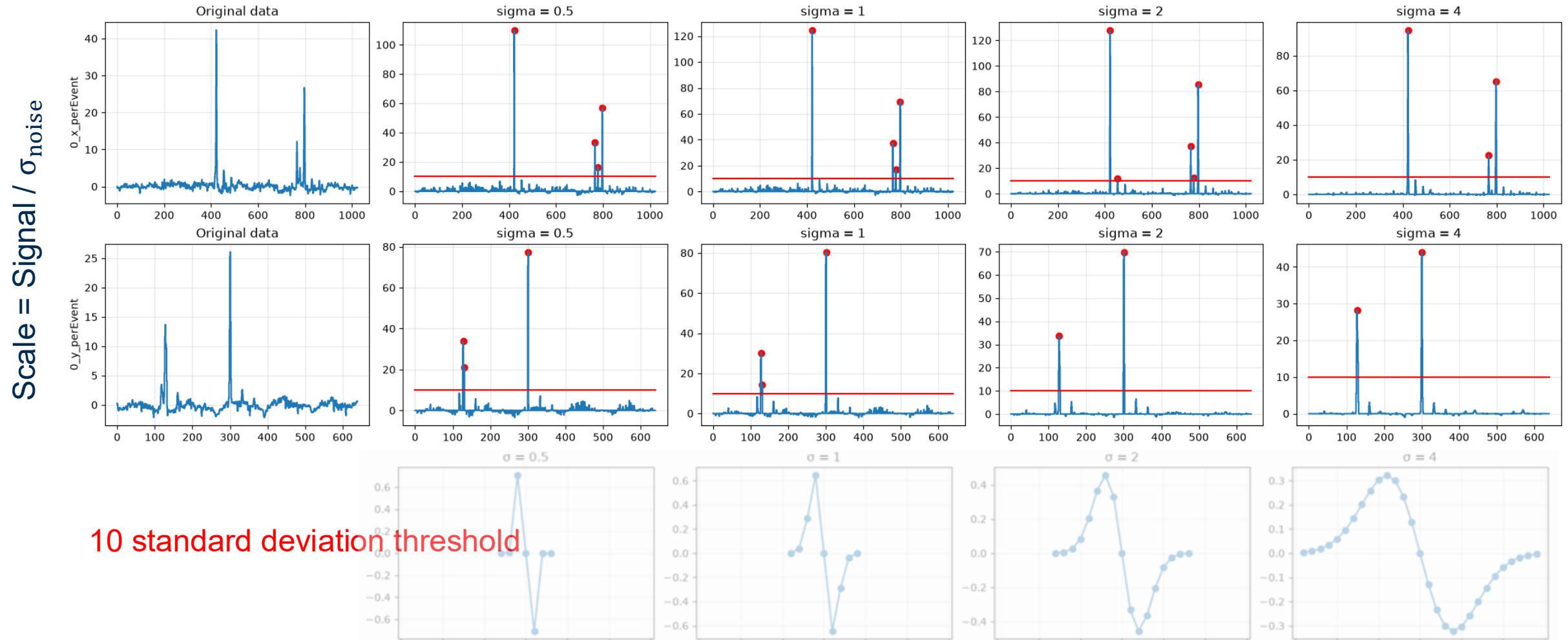
Convolute first derivative of smoothed data (y') with first derivative gaussian templates



- 5 standard deviation above mean threshold
- std and mean defined for lower 99% of values (“noise”)
- Negative outputs removed because they represent a match with reverse template



Multiply smoothed data with output of convolution

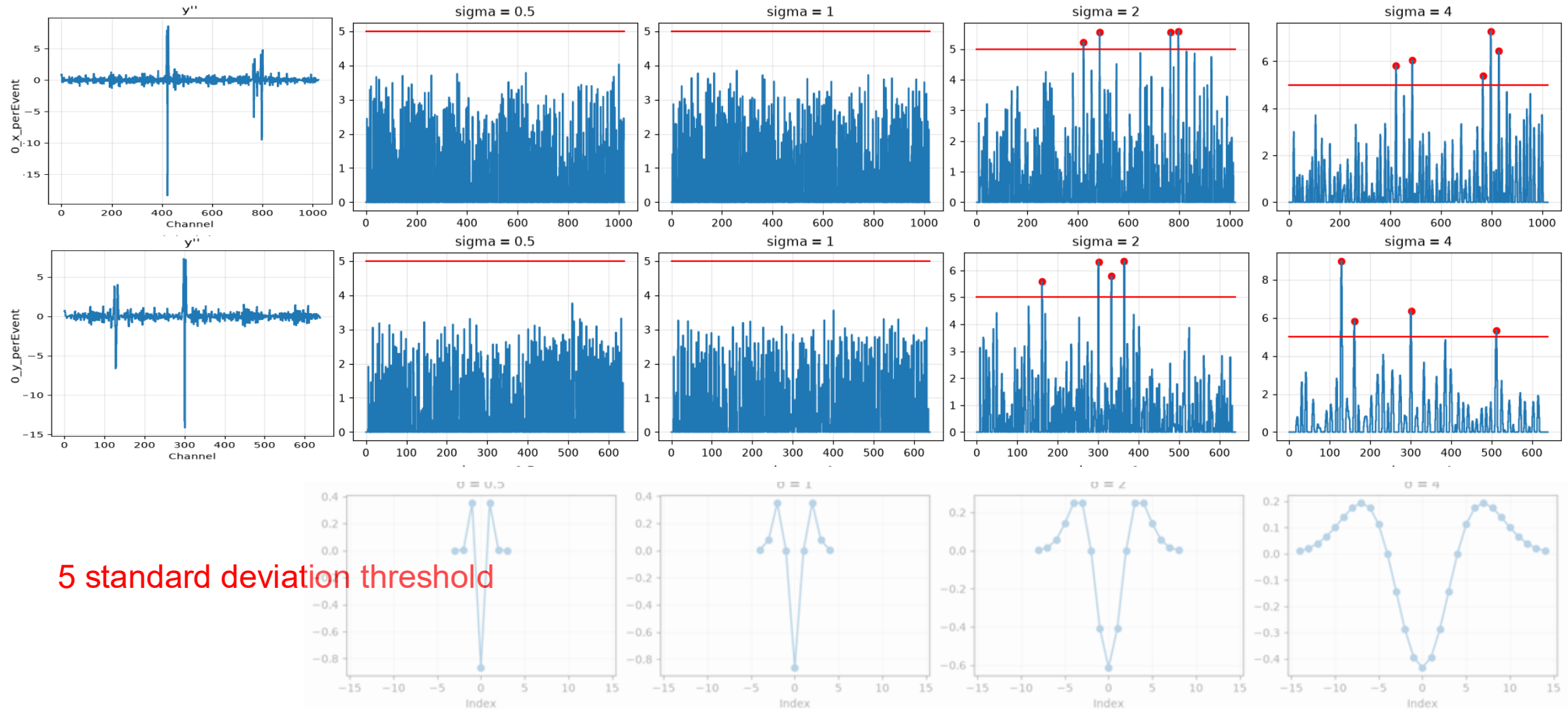


Second derivative matched filter

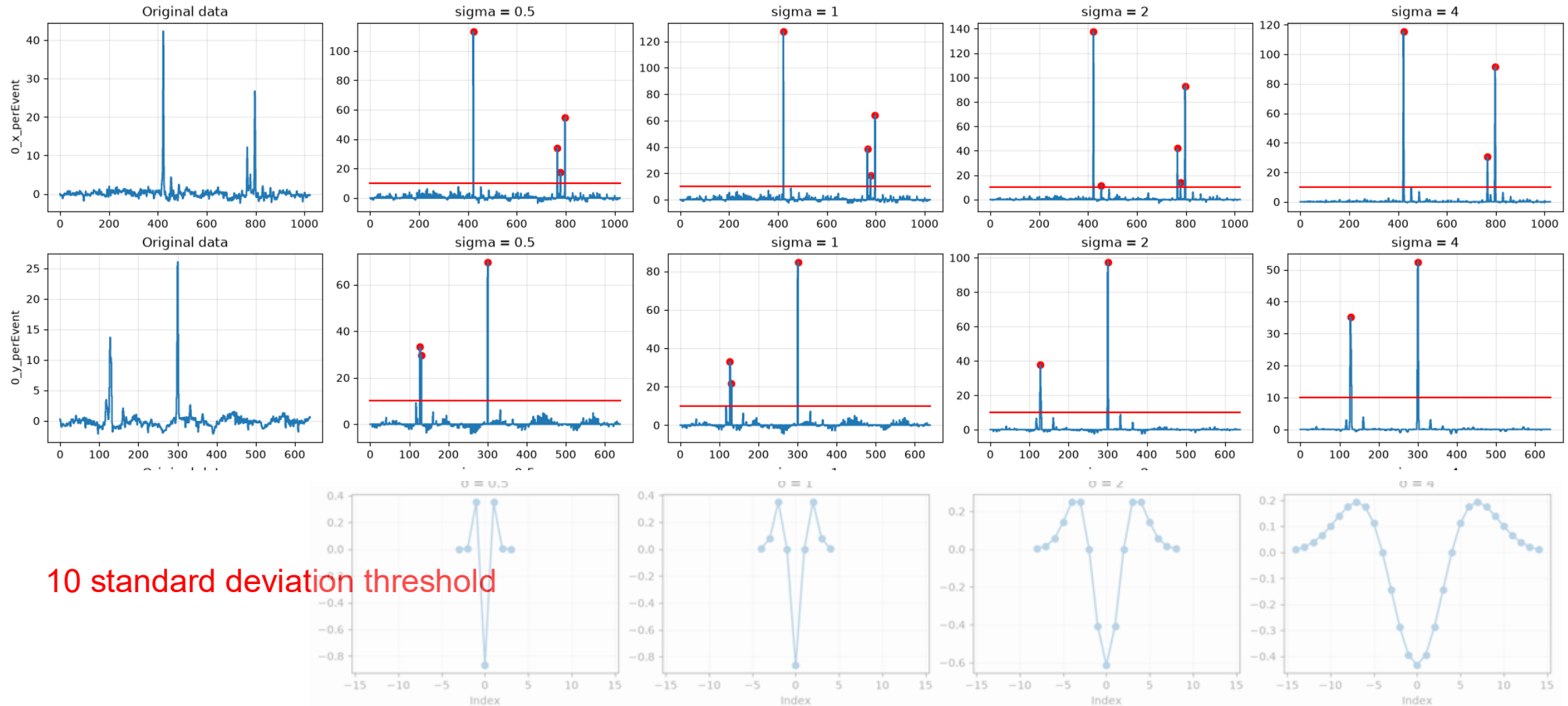
19

Convolute second derivative of smoothed data (y'') with second derivative gaussian templates

Scale = Signal / σ_{noise}



Multiply smoothed data with output of convolution



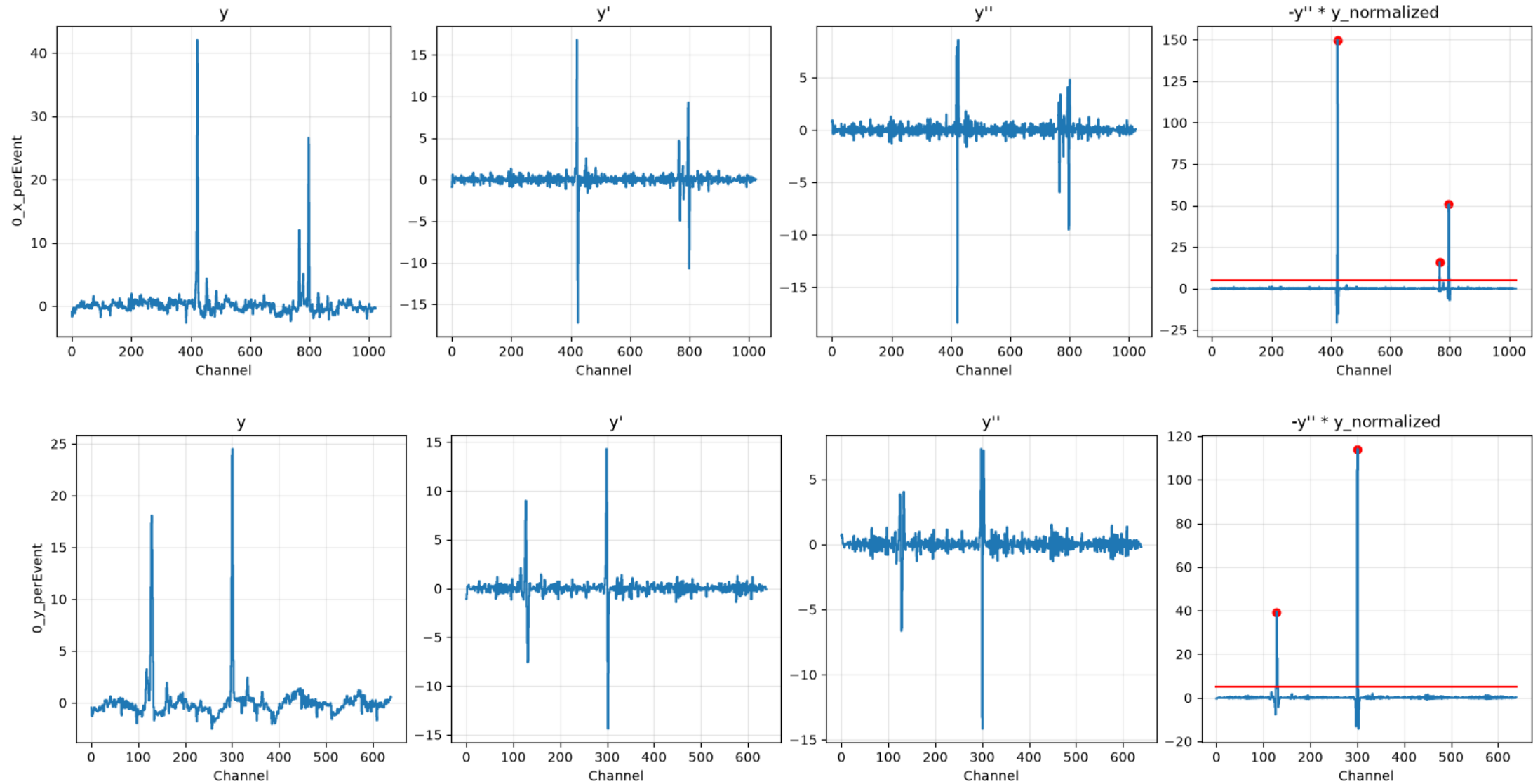
- Matched filter output does not do a great job of resolving cluster peaks on its own, but it does improve SNR when multiplied with original data
- Second derivative filter produces slightly higher SNR
- First derivative is less sensitive to noise and requires less computation
- Continuous wavelet transform
 - Apply convolution with range of template sizes
 - Takes too much time
- Real clusters most similar to $\sigma = 0.5 - 1$ templates

Derivative-based peak detection

22

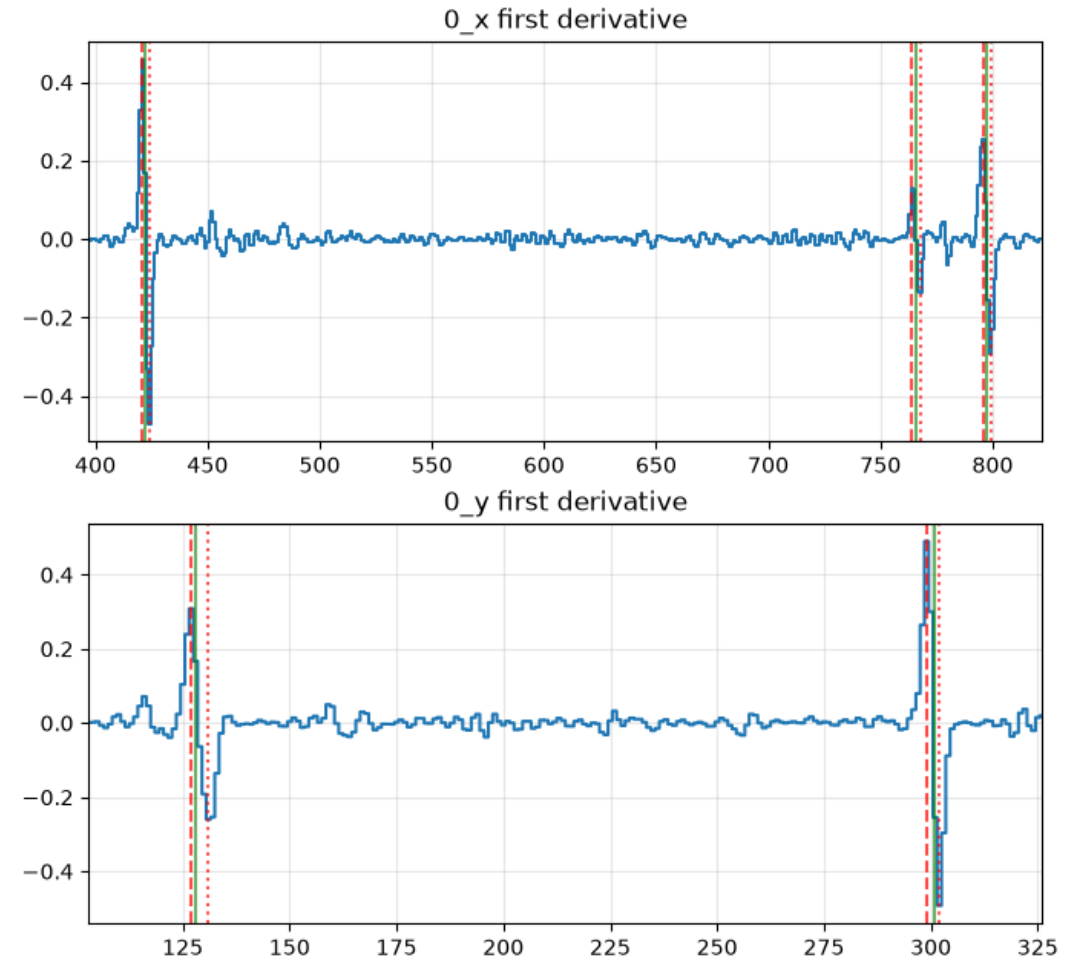
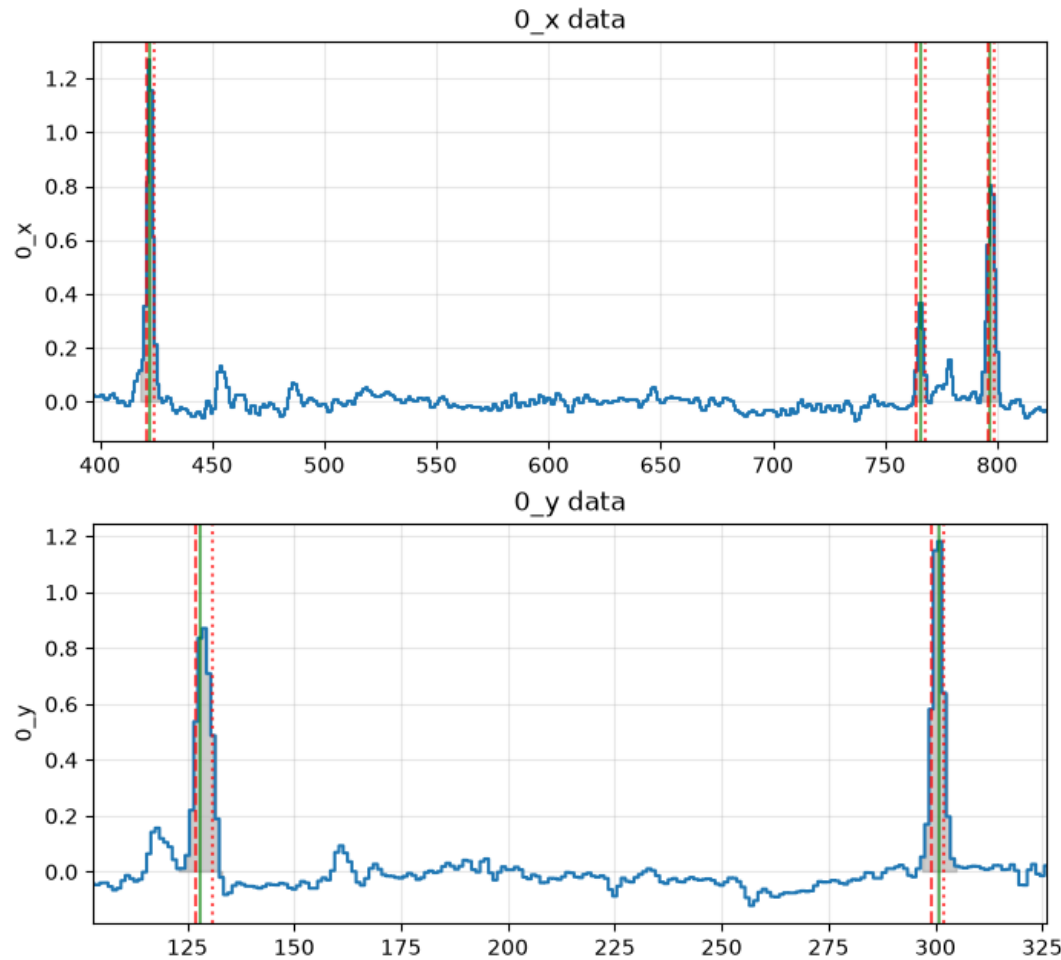
Computationally simpler. Similar SNR gain to matched filter although perhaps not as good at enhancing buried signals

Scale = Signal / σ_{noise}



5 standard deviation threshold

1. Estimate σ from nearby extrema in first derivative
2. Calculate charge within $\pm 2.756 \sigma$ of peak ($\sim 99\%$ of area)



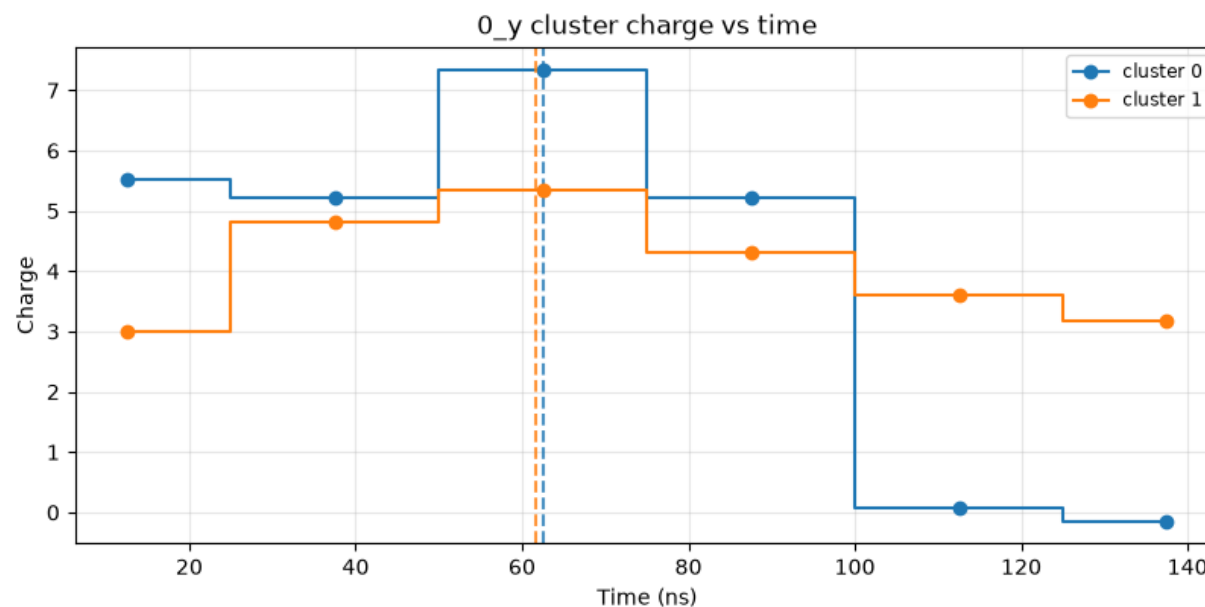
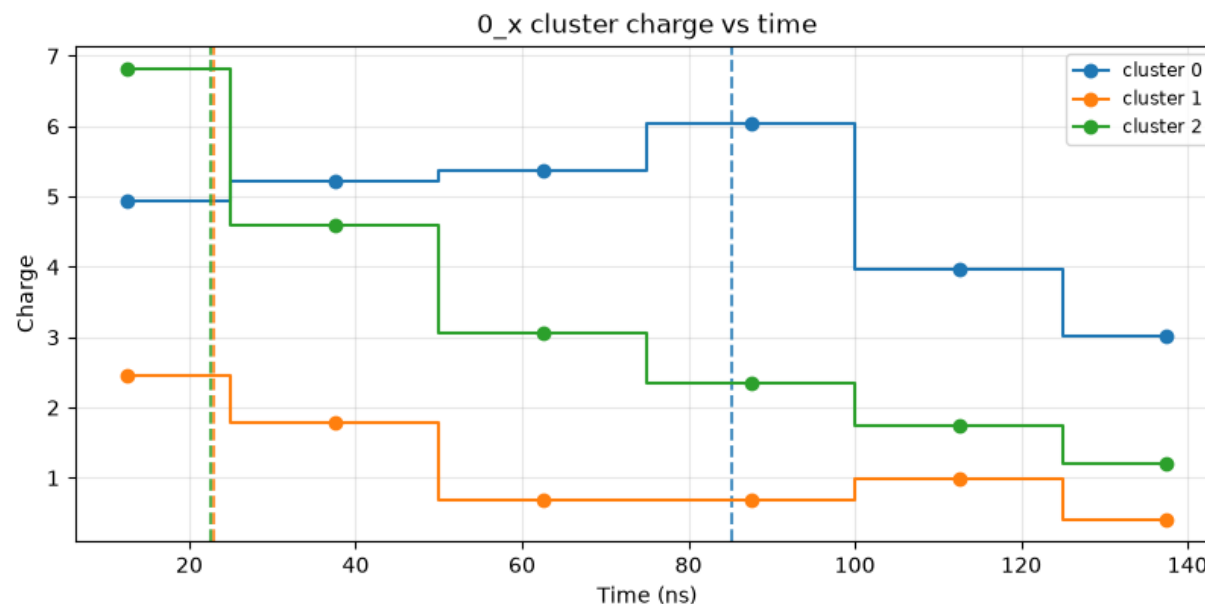
- Here, total charge is calculated using same method as before but for each time sample
 - Using same peak location and sigma determined earlier
- Dotted lines show weighted-average estimation of “cluster time”

Possible ways to match X and Y clusters

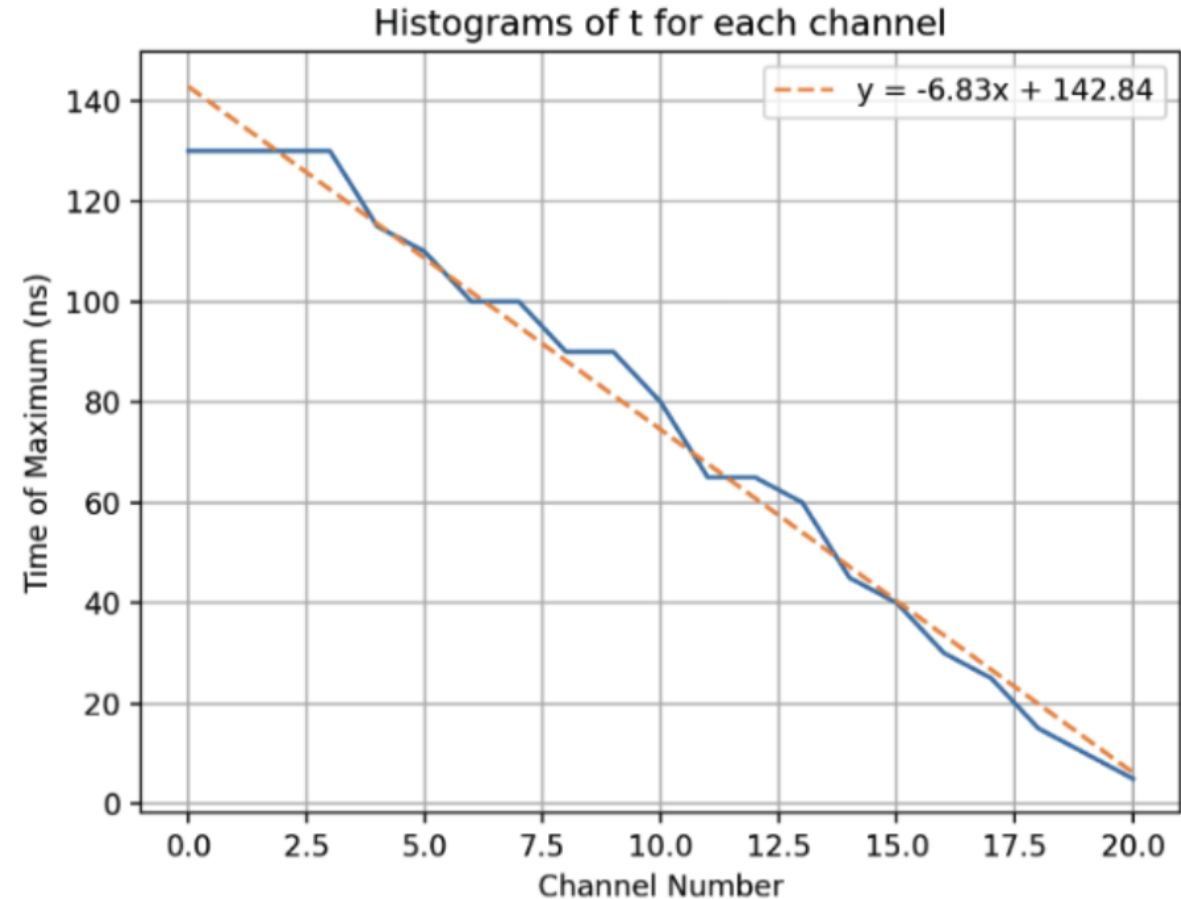
- Maximize inner product or profiles
- Minimize mean squared error between profiles
- Find correlation between total or peak charge amplitude (based on how much charge is shared between X and Y strips)

Efficiency of matching X and Y clusters will be low because of multiple hit ambiguity

- Need to use information from other detectors



- Plot time of maximum charge on each channel in cluster
- Extract angle of incidence of particle track from slope
- We may not have enough time resolution
- More difficult in events with multiple tracks
 - Charge in each 1D cluster is projection of charge accumulated from potentially several 2D hits



$$\text{Supposedly, } |\text{slope}| = (0.4\text{mm}) \cdot \frac{\cot \theta}{v_d} \sim 6.9$$

Time projection analysis

26

